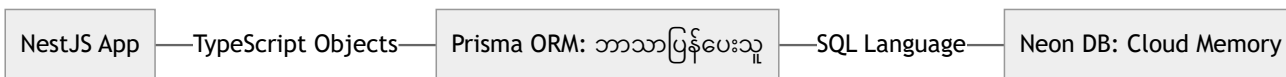


Day 2: Database Architecture & Prisma (The Ultimate Guide)

ဒီ Guide မှာ App ကို Cloud Database နဲ့ ဘယ်လိုချိတ်ဆက်ရမလဲ၊ Data Models တွေကို ဘယ်လိုဒီဇိုင်းဆွဲရမလဲ ဆိုတာတွေကို အသေးစိတ် ရှင်းပြပေးသွားပါမယ်။

The Data Flow Diagram



Step 1: Secure Configuration (.env)

File: .env

Database ရဲ့ “လိပ်စာ” (Connection String) ကို လုံခြုံစိတ်ချရတဲ့ ဖျောက်ထားတဲ့ .env ဖိုင်ထဲမှာ သိမ်းဆည်းပါမယ်။

```
DATABASE_URL="postgresql://user:pass@hostname/neondb?sslmode=require"
```

 **Deep Explainer (အသေးစိတ် ရှင်းလင်းချက်):**

ဒါကို **Environment Separation (ပတ်ဝန်းကျင် ခွဲခြားခြင်း)** လို့ ခေါ်ပါတယ်။

လျှို့ဝှက်ချက်တွေကို .env ဖိုင်ထဲမှာ သိမ်းဆည်းထားခြင်းအားဖြင့် Code တွေကို GitHub ပေါ်

တင်လိုက်တဲ့အခါ မတော်တဆ ပေါက်ကြားသွားတာမျိုး မဖြစ်အောင် ကာကွယ်ပေးနိုင်ပါတယ်။

Step 2: The Data Blueprint (schema.prisma)

File: prisma/schema.prisma

ကျွန်တော်တို့ရဲ့ Data တွေ ဘယ်လိုပုံစံ ရှိရမလဲဆိုတာကို တိတိကျကျ သတ်မှတ်ပါမယ်။

```
datasource db {  
  
  provider = "postgresql"  
  
  url = env("DATABASE_URL")  
  
}  
  
model User {  
  
  id Int @id @default(autoincrement())  
  
  email String @unique  
  
  password String  
  
  role Role @default(USER)  
  
  bookings Booking[]  
  
}  
  
model Car {  
  
  id Int @id @default(autoincrement())  
  
  brand String  
  
  model String  
  
  pricePerDay Float
```

```

bookings Booking[]

}

model Booking {

  id Int @id @default(autoincrement())

  startDate DateTime

  endDate DateTime

  userId Int

  user User @relation(fields: [userId], references: [id])

  carId Int

  car Car @relation(fields: [carId], references: [id])

}

enum Role {

  USER

  ADMIN

}

```

💡 Deep Explainer:

- **Relational Data:** Booking ဟာ User နဲ့ရော Car နဲ့ပါ

ဘယ်လိုချိတ်ဆက်ထားလဲဆိုတာ သတိပြုကြည့်ပါ။ Prisma က ဒီလို “Foreign Key”

ချိတ်ဆက်မှုတွေကို ကျွန်တော်တို့အတွက် အလိုအလျောက် စီမံပေးသွားမှာပါ။

Step 3: Synchronization (Push & Generate)

Cloud ပေါ်ကို ဒီဇိုင်းတွေရောက်သွားအောင်နဲ့ Code တွေ အဆင်သင့်ဖြစ်အောင် အောက်ပါ Command တွေကို Run ပေးပါမယ်။

```
# 1. Cloud Database Table တွေကို Update လုပ်မယ်

npx prisma db push

# 2. TypeScript "Prisma Client" ကို Generate လုပ်မယ်

npx prisma generate
```

Step 4: The Database Bridge (Prisma Service 🛠️)

File: `src/prisma/prisma.service.ts`

Database နဲ့ ချိတ်ဆက်မှုကို အမြဲရှင်သန်နေအောင် လုပ်ပေးမယ့် Service ဖြစ်ပါတယ်။

```
import { Injectable, OnModuleInit } from '@nestjs/common';

import { PrismaClient } from '@prisma/client';

@Injectable()

export class PrismaService extends PrismaClient implements OnModuleInit {

  async onModuleInit() {

    // Lifecycle Hook: App စတင်တာနဲ့ တစ်ပြိုင်နက် DB နဲ့ ချိတ်ဆက်ပါမယ်

    await this.$connect();
```

}

}

⚠️ Key Learning: Version Compatibility (ဗားရှင်း လိုက်ဖက်ညီမှု)

- ပြဿနာ (Issue): Prisma 7 ဟာ Node 22+ ကို လိုအပ်ပါတယ်။
- ဖြေရှင်းချက် (Solution): သင့်ရဲ့ Node 20 နဲ့ ကိုက်ညီအောင် Prisma 6 ကို ရွေးချယ်အသုံးပြုခဲ့ပါတယ်။
- သင်ခန်းစာ (Lesson): အသစ်ဆုံး Version တွေကို အမြဲလိုက်သုံးတာထက်၊

တည်ငြိမ်တဲ့ ပတ်ဝန်းကျင် (Stable environment) ကို တည်ဆောက်ဖို့က ပိုအရေးကြီးပါတယ်။

✅ Day 2 Graduation (ပြီးမြောက်ခြင်း)

ဘရောက်ဇာကနေ `http://localhost:3000/db-test` ကို သွားကြည့်ပါ။

userCount: 0 လို့ ပြနေတယ်ဆိုရင်တော့ Database ချိတ်ဆက်ခြင်းကို သင် အောင်မြင်စွာ တတ်မြောက်သွားပါပြီ! 🌍 🏆